

CS441: Applied ML - HW 3

▼ Part 1: CLIP: Contrastive Language-Image Pretraining

Include all the code for Part 1 in this section

▼ 1.1 Prepare data

```
import json
import os
import os.path as osp
import numpy as np
from google.colab import drive
import torch
from torchvision.datasets import Flowers102
%matplotlib inline
from matplotlib import pyplot as plt

drive.mount('/content/drive')
datadir = "/content/drive/My Drive/CS441/hw3/"

Mounted at /content/drive

# def load_flower_data(img_transform=None):
#     if os.path.isdir(datadir+ "flowers-102"):
#         do_download = False
#     else:
#         do_download = True
#     train_set = Flowers102(root=datadir, split='train', transform=img_transform, download=do_download)
#     test_set = Flowers102(root=datadir, split='val', transform=img_transform, download=do_download)
#     classes = json.load(open(osp.join(datadir, "flowers102_classes.json")))

#     return train_set, test_set, classes

def load_flower_data(img_transform=None):
    drive.mount('/content/drive')
    datadir = "." #/content/drive/My Drive/CS441/hw3/"
    if os.path.isdir(datadir+ "flowers-102"):
        do_download = False
    else:
        do_download = True
    train_set = Flowers102(root=datadir, split='train', transform=img_transform, download=do_download)
    test_set = Flowers102(root=datadir, split='val', transform=img_transform, download=do_download)
    classes = json.load(open(osp.join("/content/drive/My Drive/CS441/hw3/", "flowers102_classes.json")))

    return train_set, test_set, classes

# READ ME! This takes some time (a few minutes), so if you are using Colabs,
#     first set to use GPU: Edit->Notebook Settings->Hardware Accelerator=GPU, and restart instance

# Data structure details
#     flower_train[n][0] is the nth train image
#     flower_train[n][1] is the nth train label
#     flower_test[n][0] is the nth test image
#     flower_test[n][1] is the nth test label
#     flower_classes[k] is the name of the kth class
flower_train, flower_test, flower_classes = load_flower_data()

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
Downloading https://thor.robots.ox.ac.uk/datasets/flowers-102/102flowers.tgz to flowers-102/102flowers.tgz
100% 344862509/344862509 [00:12<00:00, 30072544.74it/s]

Extracting flowers-102/102flowers.tgz to flowers-102
Downloading https://thor.robots.ox.ac.uk/datasets/flowers-102/imagelabels.mat to flowers-102/imagelabels.mat
100% 502/502 [00:00<00:00, 33789.75it/s]

Downloading https://thor.robots.ox.ac.uk/datasets/flowers-102/setid.mat to flowers-102/setid.mat
100% 14989/14989 [00:00<00:00, 1146982.83it/s]
```

```
len(flower_train), len(flower_test)
```

```
(1020, 1020)
```

```
# Display a sample in Flowers 102 dataset
```

```
sample_idx = 0 # Choose an image index that you want to display
```

```
print("Label:", flower_classes[flower_train[sample_idx][1]])
```

```
flower_train[sample_idx][0]
```

Label: pink primrose



▼ 1.2 Prepare CLIP model

```
!pip install git+https://github.com/openai/CLIP.git
```

Looking in indexes: <https://pypi.org/simple>, <https://us-python.pkg.dev/colab-wheels/public/simple/>

Collecting git+<https://github.com/openai/CLIP.git>

Cloning <https://github.com/openai/CLIP.git> to /tmp/pip-req-build-3c294kiq

Running command git clone --filter=blob:none --quiet <https://github.com/openai/CLIP.git> /tmp/pip-req-build-3c294kiq

Resolved <https://github.com/openai/CLIP.git> to commit a9b1bf5920416aaeac965c25dd9e8f98c864f16

Preparing metadata (setup.py) ... done

Collecting ftfy

Downloading ftfy-6.1.1-py3-none-any.whl (53 kB)

53.1/53.1 KB 3.4 MB/s eta 0:00:00

Requirement already satisfied: regex in /usr/local/lib/python3.9/dist-packages (from clip==1.0) (2022.10.31)

Requirement already satisfied: tqdm in /usr/local/lib/python3.9/dist-packages (from clip==1.0) (4.65.0)

Requirement already satisfied: torch in /usr/local/lib/python3.9/dist-packages (from clip==1.0) (1.13.1+cu116)

Requirement already satisfied: torchvision in /usr/local/lib/python3.9/dist-packages (from clip==1.0) (0.14.1+cu116)

Requirement already satisfied: wcwidth>=0.2.5 in /usr/local/lib/python3.9/dist-packages (from ftfy->clip==1.0) (0.2.6)

Requirement already satisfied: typing-extensions in /usr/local/lib/python3.9/dist-packages (from torch->clip==1.0) (4.5.0)

Requirement already satisfied: requests in /usr/local/lib/python3.9/dist-packages (from torchvision->clip==1.0) (2.27.1)

Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.9/dist-packages (from torchvision->clip==1.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.9/dist-packages (from torchvision->clip==1.0) (1.22.4)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.9/dist-packages (from requests->torchvision->clip==1.0)

Requirement already satisfied: charset-normalizer~=2.0.0 in /usr/local/lib/python3.9/dist-packages (from requests->torchvision->clip==1.0)

Requirement already satisfied: urllib3<1.27,>=1.21.1 in /usr/local/lib/python3.9/dist-packages (from requests->torchvision->clip==1.0)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.9/dist-packages (from requests->torchvision->clip==1.0)

Building wheels for collected packages: clip

Building wheel for clip (setup.py) ... done

Created wheel for clip: filename=clip-1.0-py3-none-any.whl size=1369398 sha256=d586ac192a5a73e9904f78c311047fb2fb5173bd92fe

Stored in directory: /tmp/pip-ephem-wheel-cache-8s55r5bi/wheels/c8/e4/e1/11374c111387672fc2068dfbe0d4b424cb9cdd1b2e184a71b

Successfully built clip

Installing collected packages: ftfy, clip

Successfully installed clip-1.0 ftfy-6.1.1

```
import clip
```



```

from tqdm import tqdm
from torch.utils.data import DataLoader

# Load flowers dataset again. This time, with clip_preprocess as transform
flower_train_trans, flower_test_trans, flower_classes = load_flower_data(img_transform=clip_preprocess)

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

def clip_zero_shot(data_set, classes):
    data_loader = DataLoader(data_set, batch_size=64, shuffle=False) # dataloader lets you process in batch which is way faster
    # Needs code here

    return accuracy

accuracy = clip_zero_shot(data_set=flower_test_trans, classes=flower_classes)
print(f"\nAccuracy = {100*accuracy:.3f}%")

```

▼ 1.5 YOUR TASK: Test CLIP linear probe performance on Flowers 102

```

from sklearn.linear_model import LogisticRegression

"""
In this part, train a linear classifier on CLIP features
return: image features, labels in numpy format.
"""

# Creating a function called get_features.
def get_features(data_set):

    imageFeatures = [] # Creating a variable called imageFeatures.

    imageLabels = [] # Creating a variable called imageLabels.

    # Here we are using torch.no_grad().
    with torch.no_grad():

        # Here we are using a for loop to go through the images and labels.
        for images, labels in tqdm(DataLoader(data_set, batch_size=100)):

            # Creating a variable called modelFeatures to hold the encoded images from the clip model.
            modelFeatures = clip_model.encode_image(images.to(device))

            # Appending on to imageFeatures.
            imageFeatures.append(modelFeatures)

            # Appending on to imageLabels.
            imageLabels.append(labels)

    # Returning image features and image labels in numpy format.
    return torch.cat(imageFeatures).cpu().numpy(), torch.cat(imageLabels).cpu().numpy()

# Calculate the image features
train_features, train_labels = get_features(flower_train_trans)

test_features, test_labels = get_features(flower_test_trans)

# Perform logistic regression
LRC = LogisticRegression(random_state=0, C=0.316, max_iter=1000, verbose=1)

LRC.fit(train_features, train_labels)

# Evaluate using the logistic regression classifier
# Needs code here
LRCPred = LRC.predict(test_features)

accuracy = (LRCPred == test_labels).sum() / len(test_labels)

print(f"\nAccuracy = {accuracy}")

print(f"\nAccuracy = {100*accuracy:.3f}%")

```

```

100%|██████████| 11/11 [00:11<00:00, 1.08s/it]
100%|██████████| 11/11 [00:12<00:00, 1.15s/it]
[Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent workers.

Accuracy = 0.9362745098039216

Accuracy = 93.627%
[Parallel(n_jobs=1)]: Done 1 out of 1 | elapsed: 5.9s finished

```

▼ 1.6 YOUR TASK: Evaluate a nearest-neighbor classifier on CLIP features

```

# Importing the KNN model from sklearn
from sklearn.neighbors import KNeighborsClassifier

# Creating a variable called KNN that holds the KNN model.
KNN = KNeighborsClassifier(n_neighbors=21)

# Fitting the KNN model.
KNN.fit(train_features, train_labels)

KNeighborsClassifier
KNeighborsClassifier(n_neighbors=21)

# Here is the function that calculates the accuracy for each k value.
def accuracy(test_features, test_labels):
    y_pred = KNN.predict(test_features)
    return (y_pred == test_labels).sum() / len(test_labels)

# Here is the accuracy when k equals 1.
accuracy(test_features, test_labels)

0.8411764705882353

# Here is the accuracy when k equals 3.
accuracy(test_features, test_labels)

0.8313725490196079

# Here is the accuracy when k equals 5.
accuracy(test_features, test_labels)

0.846078431372549

# Here is the accuracy when k equals 11.
accuracy(test_features, test_labels)

0.85

# Here is the accuracy when k equals 21.
accuracy(test_features, test_labels)

0.7980392156862746

```

▼ Part 2: Fine-Tune for Pets Image Classification

Include all the code for Part 2 in this section

▼ 2.1 Prepare Data

```

import torch
import torch.nn as nn
import torch.optim.lr_scheduler as lrs
from torch.utils.data import DataLoader
import torchvision
from torchvision import datasets
from torchvision import transforms

```

```

import matplotlib.pyplot as plt
from tqdm import tqdm

import os
from pathlib import Path
import numpy as np

# Mount and define data dir
from google.colab import drive
drive.mount('/content/drive')
datadir = "/content/"
save_dir = "/content/drive/My Drive/CS441/hw3"

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

def load_pet_dataset(train_transform = None, test_transform = None):
    OxfordIIITPet = datasets.OxfordIIITPet
    if os.path.isdir(datadir+ "oxford-iiit-pet"):
        do_download = False
    else:
        do_download = True
    training_set = OxfordIIITPet(root = datadir,
                                split = 'trainval',
                                transform = train_transform,
                                download = do_download)

    test_set = OxfordIIITPet(root = datadir,
                             split = 'test',
                             transform = test_transform,
                             download = do_download)
    return training_set, test_set

train_set, test_set = load_pet_dataset()

# Display a sample in OxfordIIITPet dataset
sample_idx = 0 # Choose an image index that you want to display
print("Label:", train_set.classes[train_set[sample_idx][1]])
train_set[sample_idx][0]

```

```

Downloading https://thor.robots.ox.ac.uk/datasets/pets/images.tar.gz to /content/oxford-iiit-pet/images.tar.gz
100% 791918971/791918971 [00:26<00:00, 28747845.44it/s]
Extracting /content/oxford-iiit-pet/images.tar.gz to /content/oxford-iiit-pet
Downloading https://thor.robots.ox.ac.uk/datasets/pets/annotations.tar.gz to /content/oxford-iiit-pet/annotations.tar.gz
100% 19173078/19173078 [00:01<00:00, 24573412.31it/s]
Extracting /content/oxford-iiit-pet/annotations.tar.gz to /content/oxford-iiit-pet
Label: Abyssinian

```



▼ 2.2 Data Preprocess

```

from torchvision import transforms
from torch.utils.data import DataLoader

# Feel free to add augmentation choices

# Apply data augmentation
train_transform = transforms.Compose([
    transforms.Resize(224),
    transforms.RandomHorizontalFlip(p = 0.10),
    transforms.RandomVerticalFlip(p = 0.10),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std= [0.229, 0.224, 0.225]),
])

test_transform = transforms.Compose([
    transforms.Resize(224), # resize to 224x224 because that's the size of ImageNet images
    transforms.RandomHorizontalFlip(p = 0.10),
    transforms.RandomVerticalFlip(p = 0.10),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std= [0.229, 0.224, 0.225]),
])

# Feel free to change
train_set, test_set = load_pet_dataset(train_transform, test_transform)
train_loader = DataLoader(dataset=train_set,
                           batch_size=64,
                           shuffle=True,
                           num_workers=2)

test_loader = DataLoader(dataset=test_set,
                           batch_size=64,
                           shuffle=False,
                           num_workers=2)

```

▼ 2.3 Helper Functions

```

# Display the number of parameters and model structure
def display_model(model):
    # Check number of parameters
    summary_dict = {}
    num_params = 0
    summary_str = [' '*80]

    for module_name, module in model.named_children():
        summary_count = 0
        for name, param in module.named_parameters():
            if param.requires_grad:
                summary_count += param.numel()
                num_params += param.numel()
        summary_dict[module_name] = [summary_count]
        summary_str+= [f'- {module_name: <40} : {str(summary_count):^34s}']

    summary_dict['total'] = [num_params]

    # print summary string
    summary_str += [' '*80]
    summary_str += ['--' + f'{"Total":<40} : {str(num_params) + " params":^34s}' + '--']
    print('\n'.join(summary_str))

    # print model structure
    print(model)

```

```
# Plot loss or accuracy
def plot_losses(train, val, test_frequency, num_epochs):
    plt.plot(train, label="train")
    indices = [i for i in range(num_epochs) if ((i+1)%test_frequency == 0 or i == 0 or i == 1)]
    plt.plot(indices, val, label="val")
    plt.title("Loss Plot")
    plt.ylabel("Loss")
    plt.xlabel("Epoch")
    plt.legend()
    plt.show()

def plot_accuracy(train, val, test_frequency, num_epochs):
    indices = [i for i in range(num_epochs) if ((i+1)%test_frequency == 0 or i == 0 or i == 1)]
    plt.plot(indices, train, label="train")
    plt.plot(indices, val, label="val")
    plt.title("Training Plot")
    plt.ylabel("Accuracy")
    plt.xlabel("Epoch")
    plt.legend()
    plt.show()

def save_checkpoint(save_dir, model, save_name = 'best_model.pth'):
    save_path = os.path.join(save_dir, save_name)
    torch.save(model.state_dict(), save_path)

def load_model(model, save_dir, save_name = 'best_model.pth'):
    save_path = os.path.join(save_dir, save_name)
    model.load_state_dict(torch.load(save_path))
    return model
```

2.4 YOUR TASK: Fine-Tune Pre-trained Network on Pets

Read and understand the code and then uncomment it. Then, set up your learning rate, learning scheduler, and train/evaluate. Adjust as necessary to reach target performance.

```
def train(train_loader, model, criterion, optimizer):
    """
    Train network
    :param train_loader: training dataloader
    :param model: model to be trained
    :param criterion: criterion used to calculate loss (should be CrossEntropyLoss from torch.nn)
    :param optimizer: optimizer for model's params (Adams or SGD)
    :return: mean training loss
    """
    model.train()
    loss_ = 0.0
    losses = []

    # TO DO: read this documentation and then uncomment the line below; https://pypi.org/project/tqdm/
    it_train = tqdm(enumerate(train_loader), total=len(train_loader), desc="Training ...", position = 0) # progress bar
    for i, (images, labels) in it_train:

        # TO DO: read/understand and then uncomment these lines
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        prediction = model(images)
        loss = criterion(prediction, labels)
        it_train.set_description(f'loss: {loss:.3f}')
        loss.backward()
        optimizer.step()
        losses.append(loss)

    return torch.stack(losses).mean().item()

def test(test_loader, model, criterion):
    """
    Test network.
    :param test_loader: testing dataloader
    :param model: model to be tested
    :param criterion: criterion used to calculate loss (should be CrossEntropyLoss from torch.nn)
    :return: mean_accuracy: mean accuracy of predicted labels
            test_loss: mean test loss during testing
    """
    model.eval()
```



```

losses = []
correct = 0
total = 0

# TO DO: read this documentation and then uncomment the line below; https://pypi.org/project/tqdm/
it_test = tqdm(enumerate(test_loader), total=len(test_loader), desc="Validating ...", position = 0)
for i, (images, labels) in it_test:

    # TO DO: read/understand and then uncomment these lines
    images, labels = images.to(device), labels.to(device)
    with torch.no_grad(): # https://pytorch.org/docs/stable/generated/torch.no_grad.html
        output = model(images)
        preds = torch.argmax(output, dim=-1)
        loss = criterion(output, labels)
        losses.append(loss.item())
        correct += (preds == labels).sum().item()
        total += len(labels)

mean_accuracy = correct / total
test_loss = np.mean(losses)
print('Mean Accuracy: {0:.4f}'.format(mean_accuracy))
print('Avg loss: {}'.format(test_loss))

return mean_accuracy, test_loss

import warnings
warnings.filterwarnings("ignore", category=UserWarning)

device = 'cuda'
# loads a pre-trained ResNet-34 model
model = torch.hub.load('pytorch/vision:v0.10.0', 'resnet34', pretrained=True)
target_class = 37
# TO DO: replace the last layer with a new linear layer for Pets classification
model.fc = nn.Linear(512, target_class)
model = model.to(device)
display_model(model) # displays the model structure and parameter count

```

100% 83.3M/83.3M [00:02<00:00, 41.9MB/s]

<https://colab.research.google.com/drive/1bglyRMzNIANKpNMTVusD8uIW5Vnc5ZAI?authuser=1#scrollTo=yHqQKYezZvz5&printMode=true> 10/15

```

(Conv2d).conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False,
               ...
               ...

# Training Setting. Feel free to change.
num_epochs = 20
test_interval = 5

# TO DO: set initial learning rate
learn_rate = 0.0001
optimizer = torch.optim.Adam(model.parameters(), lr=learn_rate)

# TO DO: define your learning rate scheduler, e.g. StepLR
# https://pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.StepLR.html#torch.optim.lr_scheduler.StepLR
lr_scheduler = lrs.StepLR(optimizer, step_size=5, gamma=0.1, last_epoch=-1, verbose=False)

criterion = torch.nn.CrossEntropyLoss()

train_losses = []
train_accuracy_list = []
test_losses = []
test_accuracy_list = []

# Iterate over the DataLoader for training data
for epoch in tqdm(range(num_epochs), total=num_epochs, desc="Training ...", position=1):
    train_loss = train(train_loader, model, criterion, optimizer) # Train the Network for one epoch
    # TO DO: uncomment the line below. It should be called each epoch to apply the lr_scheduler
    lr_scheduler.step()
    train_losses.append(train_loss)
    print(f'Loss for Training on epoch {str(epoch)} is {str(train_loss)} \n')

    if(epoch%test_interval==0 or epoch==1 or epoch==num_epochs-1):
        print('Evaluating Network')

        train_accuracy, _ = test(train_loader, model, criterion) # Get training accuracy
        train_accuracy_list.append(train_accuracy)

        print(f'Training accuracy on epoch {str(epoch)} is {str(train_accuracy)} \n')

        test_accuracy, test_loss = test(test_loader, model, criterion) # Get testing accuracy and error
        test_losses.append(test_loss)
        test_accuracy_list.append(test_accuracy)

        print(f'Testing accuracy on epoch {str(epoch)} is {str(test_accuracy)} \n')

# Checkpoints are used to save the model with best validation accuracy
if test_accuracy >= max(test_accuracy_list):
    print("Saving Model")
    save_checkpoint(save_dir, model, save_name = 'best_model.pth') # Save model with best performance

```



```

Training ...: 85%|██████████| 17/20 [13:04<02:09, 43.08s/it]Loss for Training on epoch 16 is 0.0376475022864342
loss: 0.049: 100%|██████████| 58/58 [00:29<00:00, 1.97it/s]

Training ...: 90%|██████████| 18/20 [13:34<01:18, 39.01s/it]Loss for Training on epoch 17 is 0.04210372641682625
loss: 0.015: 100%|██████████| 58/58 [00:31<00:00, 1.85it/s]

Training ...: 95%|██████████| 19/20 [14:05<00:36, 36.75s/it]Loss for Training on epoch 18 is 0.033582963049411774
loss: 0.036: 100%|██████████| 58/58 [00:29<00:00, 1.96it/s]Loss for Training on epoch 19 is 0.03203754872083664

Evaluating Network

Validating ...: 100%|██████████| 58/58 [00:26<00:00, 2.21it/s]Mean Accuracy: 0.9959
Avg loss: 0.02015137140125293
Training accuracy on epoch 19 is 0.9959239130434783

Validating ...: 100%|██████████| 58/58 [00:27<00:00, 2.13it/s]

Training ...: 100%|██████████| 20/20 [15:29<00:00, 46.46s/it]Mean Accuracy: 0.9000
Avg loss: 0.3439024358602433
Testing accuracy on epoch 19 is 0.8999727446170619

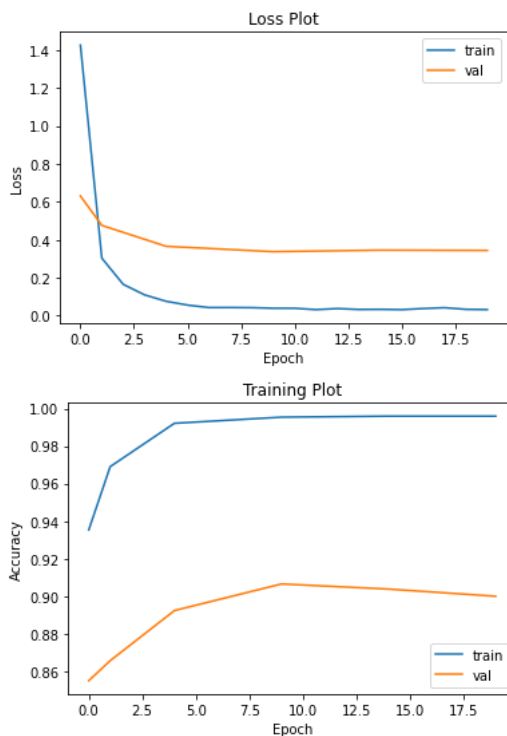
```

▼ 2.5 Plotting of losses and accuracy

```

plot_losses(train_losses, test_losses, test_interval, num_epochs)
plot_accuracy(train_accuracy_list, test_accuracy_list, test_interval, num_epochs)

```



▼ 2.6 Evaluating trained model

```

# TO DO: initialize your trained model as you did before so that you can load the parameters into it
model = torch.hub.load('pytorch/vision:v0.10.0', 'resnet34', pretrained=True).to(device)
# replace last layer
model.fc = nn.Linear(512, target_class)

load_model(model, save_dir) # Load the trained weight

model = model.to(device)

```

```
test_accuracy, test_loss= test(test_loader, model, criterion)
print(f"Testing accuracy is {str(test_accuracy)} \n")

Using cache found in /root/.cache/torch/hub/pytorch_vision_v0.10.0
Validating ...: 100%|██████████| 58/58 [00:27<00:00, 2.11it/s]Mean Accuracy: 0.9027
Avg loss: 0.33241005807087337
Testing accuracy is 0.902698282910875
```

Part 3: No coding for this part

▼ Part 4: Stretch Goals

Include any new code needed for Part 3 here

```
! pip install transformers[BPE]

Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting transformers[BPE]
  Downloading transformers-4.27.3-py3-none-any.whl (6.8 MB)
  6.8/6.8 MB 51.2 MB/s eta 0:00:00
WARNING: transformers 4.27.3 does not provide the extra 'bpe'
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.9/dist-packages (from transformers[BPE]) (1.22.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.9/dist-packages (from transformers[BPE]) (23.0)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.9/dist-packages (from transformers[BPE]) (2022.10.31)
Requirement already satisfied: requests in /usr/local/lib/python3.9/dist-packages (from transformers[BPE]) (2.27.1)
Collecting tokenizers!=0.11.3,<0.14,>=0.11.1
  Downloading tokenizers-0.13.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (7.6 MB)
  7.6/7.6 MB 92.4 MB/s eta 0:00:00
Collecting huggingface-hub<1.0,>=0.11.0
  Downloading huggingface_hub-0.13.3-py3-none-any.whl (199 kB)
  199.8/199.8 KB 26.0 MB/s eta 0:00:00
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.9/dist-packages (from transformers[BPE]) (6.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.9/dist-packages (from transformers[BPE]) (3.10.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.9/dist-packages (from transformers[BPE]) (4.65.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.9/dist-packages (from huggingface-hub<1.0,>=0.11.0) (4.5.0)
Requirement already satisfied: charset-normalizer<=2.0.0 in /usr/local/lib/python3.9/dist-packages (from requests->transformers[BPE]) (2.0.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.9/dist-packages (from requests->transformers[BPE]) (2022.12.7)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /usr/local/lib/python3.9/dist-packages (from requests->transformers[BPE]) (1.26.13)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.9/dist-packages (from requests->transformers[BPE]) (3.4)
Installing collected packages: tokenizers, huggingface-hub, transformers
Successfully installed huggingface-hub-0.13.3 tokenizers-0.13.2 transformers-4.27.3

! wget https://s3.amazonaws.com/research.metamind.io/wikitext/wikitext-2-raw-v1.zip
! unzip wikitext-2-raw-v1.zip

--2023-03-24 01:25:41-- https://s3.amazonaws.com/research.metamind.io/wikitext/wikitext-2-raw-v1.zip
Resolving s3.amazonaws.com (s3.amazonaws.com)... 52.217.230.224, 52.217.95.80, 54.231.170.160, ...
Connecting to s3.amazonaws.com (s3.amazonaws.com)|52.217.230.224|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4721645 (4.5M) [application/zip]
Saving to: 'wikitext-2-raw-v1.zip'

wikitext-2-raw-v1.z 100%[=====>] 4.50M 19.7MB/s in 0.2s

2023-03-24 01:25:42 (19.7 MB/s) - 'wikitext-2-raw-v1.zip' saved [4721645/4721645]

Archive: wikitext-2-raw-v1.zip
  creating: wikitext-2-raw/
  inflating: wikitext-2-raw/wiki.test.raw
  inflating: wikitext-2-raw/wiki.valid.raw
  inflating: wikitext-2-raw/wiki.train.raw

# Here we are importing the required libraries for tokenizations.
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.models import WordPiece
from tokenizers.pre_tokenizers import Whitespace
from tokenizers.trainers import BpeTrainer
from tokenizers.trainers import WordPieceTrainer

# Here we are getting BPE tokenizer.
tokenizerBPE = Tokenizer(BPE())
tokenizerBPE.pre_tokenizer = Whitespace()
```

```
# Here we are tokenizing two different sentences using the BPE tokenizer.
trainerBPE = BpeTrainer(special_tokens=["[UNK]", "[CLS]", "[SEP]", "[PAD]", "[MASK]"])
tokenizerBPE.train(files=[f"wikitext-2-raw/wiki.{split}.raw" for split in ["test", "train", "valid"]], trainer=trainerBPE)
outputBPE1 = tokenizerBPE.encode("I am learning about word tokenizers. They are not very complicated, and they are a good way to c
outputBPE2 = tokenizerBPE.encode("This is the second sentence that I am using to report the tokenizations. Hopefully the encodings w
print(outputBPE1.tokens)
print(outputBPE2.tokens)

# Here we are getting WordPiece tokenizer.
tokenizerWP = Tokenizer(WordPiece())
tokenizerWP.pre_tokenizer = Whitespace()

# Here we are tokenizing two different sentences using the WordPiece tokenizer.
trainerWP = WordPieceTrainer(special_tokens=["[UNK]", "[CLS]", "[SEP]", "[PAD]", "[MASK]"])
tokenizerWP.train(files=[f"wikitext-2-raw/wiki.{split}.raw" for split in ["test", "train", "valid"]], trainer=trainerWP)
outputWP1 = tokenizerWP.encode("I am learning about word tokenizers. They are not very complicated, and they are a good way to cor
outputWP2 = tokenizerWP.encode("This is the second sentence that I am using to report the tokenizations. Hopefully the encodings w
print(outputWP1.tokens)
print(outputWP2.tokens)

['I', 'am', 'learning', 'about', 'word', 'to', 'ken', 'izers', '.', 'They', 'are', 'not', 'very', 'complicated', ',', 'and',
['This', 'is', 'the', 'second', 'sentence', 'that', 'I', 'am', 'using', 'to', 'report', 'the', 'to', 'ken', 'izations', '.',
['I', 'am', 'learning', 'about', 'word', 'to', '##ken', '##izers', '.', 'They', 'are', 'not', 'very', 'complicated', ',', 'ar
['This', 'is', 'the', 'second', 'sentence', 'that', 'I', 'am', 'using', 'to', 'report', 'the', 'to', '##ken', '##izations',

# example network definition that needs to be modified for custom network stretch goal

class Network(nn.Module):
    def __init__(self, num_classes=10, dropout = 0.5):
        super(Network, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 256, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )

        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(p=dropout),
            nn.Linear(256 * 6 * 6, 512),
            nn.ReLU(inplace=True),
            nn.Dropout(p=dropout),
            nn.Linear(512, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, num_classes),
        )

    def forward(self, x):
        N, c, H, W = x.shape
        features = self.features(x)
        pooled_features = self.avgpool(features)
        output = self.classifier(torch.flatten(pooled_features, 1))
        return output
```

✓ 18s completed at 8:26 PM

● ×